

A large, patterned snake is coiled around a black office chair in a dark, misty jungle. The snake's head is raised, and its tongue is flicking out. In the background, a computer monitor sits on a wooden desk, displaying a 2D maze game with a character and various items. The jungle is filled with large green leaves, hanging vines, and a small fire burning in a metal container on the right. The overall atmosphere is mysterious and surreal.

Lecture 5: Pygame Review and 2-D Lists

Virtual Environments

- Self-contained directory
- Isolates specific Python version
- Isolates Python packages (pygame, etc.)
- Doesn't pollute system-wide Python or packages
- Use different Python version per project
- Use different packages per project
- Use different package versions per project

Install Python Environments Extension (Pre-Release)

The screenshot shows the Visual Studio Code interface with the Python Environments extension details page. The extension is by Microsoft and is currently in a pre-release state. The interface includes a sidebar with the extension's details, a main panel with the extension's description and installation instructions, and a right sidebar showing the installed extensions.

Python Environments (preview)

Note: The Python Environments icon may no longer appear in the Activity Bar due to the ongoing rollout of the Python Environments extension. To restore the extension, add `"python.useEnvironmentsExtension": true` to your User settings. This setting is temporarily necessary until the rollout is complete!

Installation

Identifier	ms-python.python-vscode-python-envs
Version	1.11.12792136
Last Updated	2025-10-07, 05:30:01
Size	7.80MB
Cache	8.29KB

Marketplace

Publish...	2024-12-10, 14:22:05
Last Released	2025-10-06, 18:12:14

Categories

Other

Python Environments (preview)

Provides a unified python environment experience

Overview

The Python Environments extension for VS Code helps you manage Python environments and packages using your preferred environment manager, backed by its extensible APIs. This extension provides unique support for specifying environments for specific files, entire Python folders, or projects, including multi-root and mono-repo scenarios. The core feature set includes:

- Create, delete, and manage environments
- Install and uninstall packages within the selected environment
- Create activated terminals

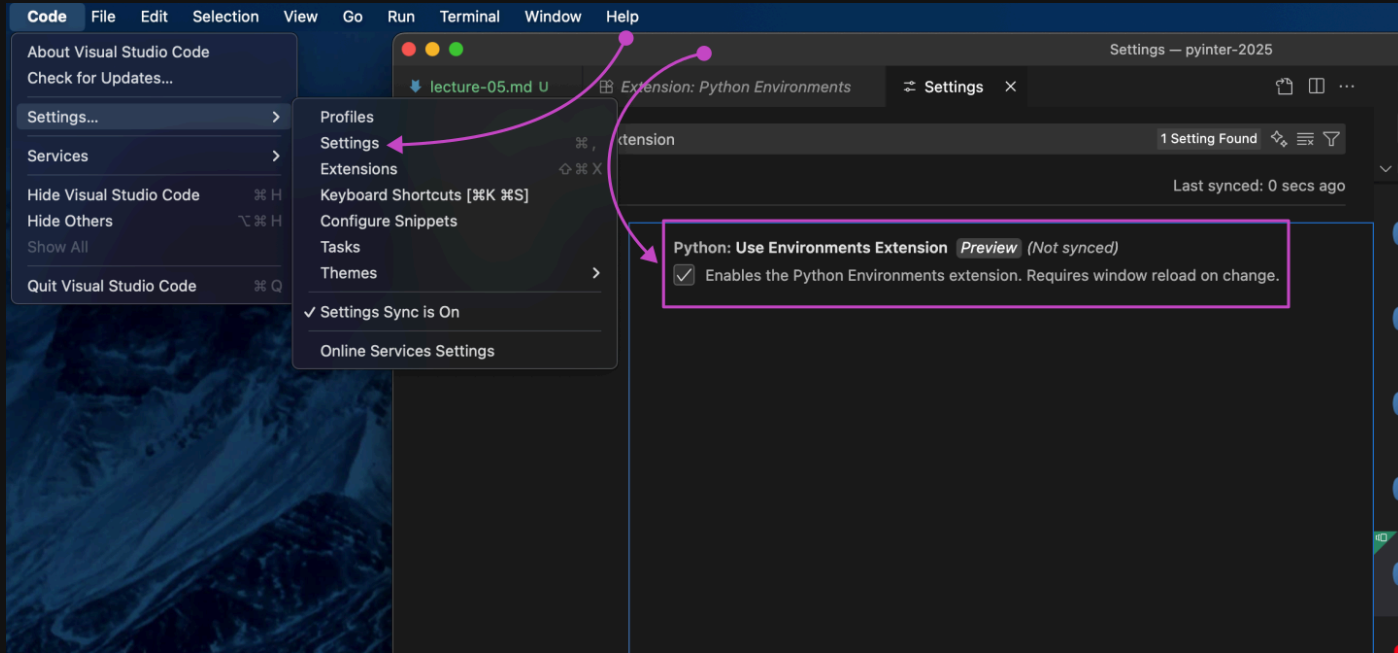
EXTENSIONS

Search Extensions in Marketplace

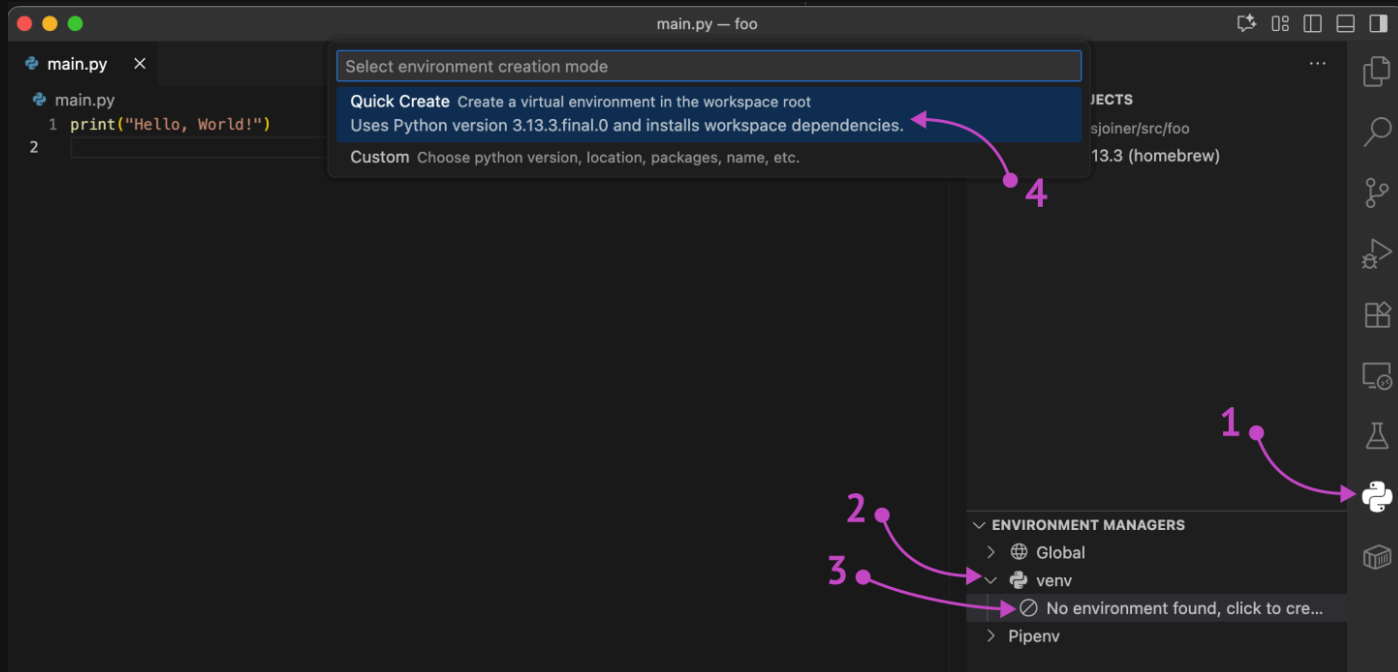
INSTALLED 24

- Pylance** (259ms) - A performant, feature-rich language server for Python in VS C... (Microsoft)
- Pylint** (17ms) - Linting support for Python files using Pylint. (Microsoft)
- Python** (84ms) - Python language support with extension access points for Intel... (Microsoft)
- Python Debugger** (24ms) - Python Debugger extension using debugpy. (Microsoft)
- Python Environments** (23ms) - Provides a unified python environment experience (Microsoft)
- Reinbow CSV** (5ms) - Highlight CSV and TSV files, Run SQL-like queries mechatroner
- Serial Monitor** - Send and receive text from serial ports. (Microsoft)
- Vim** (36ms) - Vim emulation for Visual Studio Code (vscodevim)
- VS Code Speech**

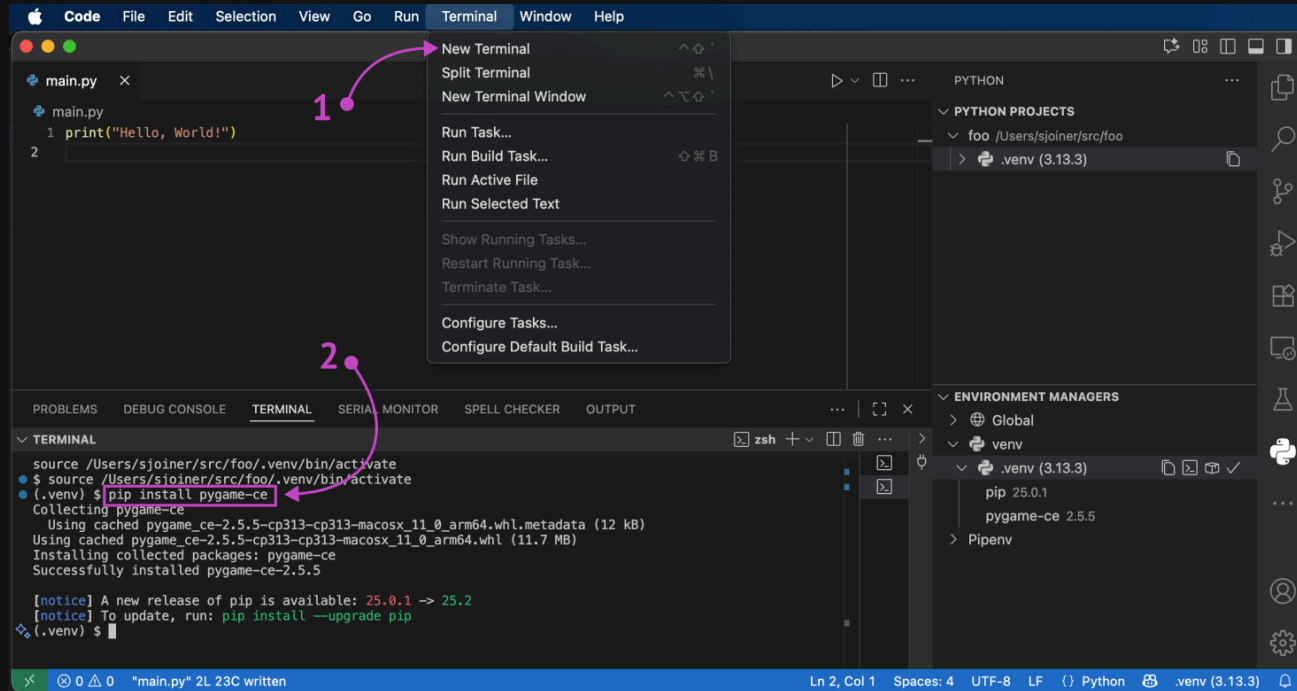
Enable Use Environments Extension



Create Virtual Environment



Install Pygame in Your Virtual Environment



- Prefer `pygame-ce` over `pygame`
- Better performance, bug fixes
- Docs: <https://pyga.me/docs/>

Hello, Pygame!

- `import pygame`
 - Imports the Pygame module
 - Call Pygame methods with `pygame` variable
- `pygame.init()`
 - Initializes Pygame
- `pygame.quit()`
 - Quits Pygame
- `print` still works
 - Useful for debugging
- Where's the screen?

```
import pygame

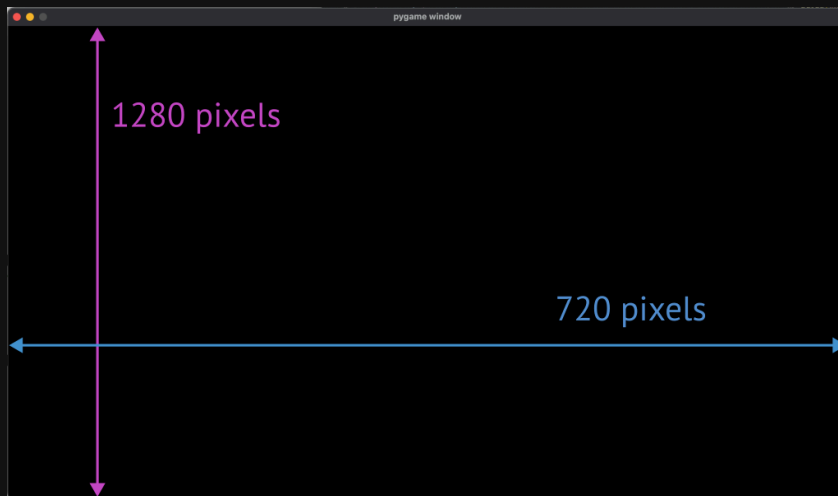
pygame.init()
print("Hello, World!")
pygame.quit()
```

🐍 Creating a Screen

- Remove `print("Hello, World!")`
- Call `pygame.display.set_mode`
 - Initializes our screen
 - Takes tuple argument
 - `(width, height)`
- Program ends right after our screen appears
- Let's fix that...

```
import pygame

pygame.init()
screen = pygame.display.set_mode((1280, 720))
pygame.quit()
```



Run Until User Quits

- `while running :`
 - "Main loop"
 - All of our game code goes here
 - Loops until we set `running` to False
- `pygame.event.get()`
 - Gets *events* since last call
 - More on this next...
- `for event in events:`
 - "Event loop"
 - Handle *events*
 - Quit when `pygame.QUIT` event seen
 - User clicks "x" on window
- Try it!

```
import pygame

pygame.init()
screen = pygame.display.set_mode((1280, 720))
running = True

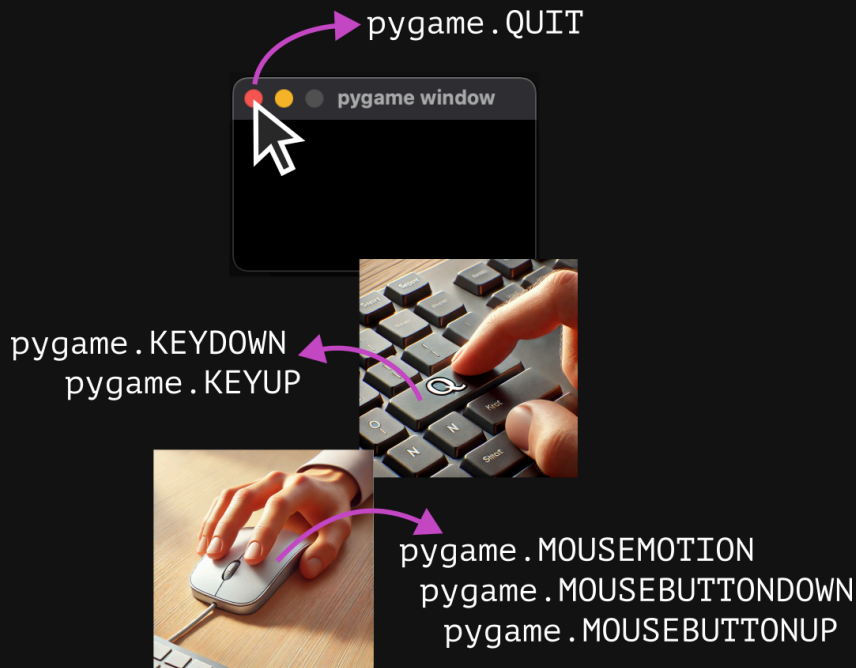
while running:
    events = pygame.event.get()
    for event in events:
        if event.type == pygame.QUIT:
            running = False

pygame.quit()
```

- But what are events? ...

Pygame Events

- Occur when "things happen"
 - Press a key on the keyboard
 - Click a mouse button
 - Move the mouse
 - Click the "x" to close the window
 - Lots more...
- Events are "fired" or "emitted"
- Events have associated data
- Example: `pygame.KEYDOWN` data
 - `key` : value of key that was pressed
 - `mod` : state of keyboard modifiers
 - Shift, Ctrl, etc.



Pygame event documentation:

<https://www.pygame.org/docs/ref/event.html>



pygame.KEYDOWN Event

```
import pygame

pygame.init()
screen = pygame.display.set_mode((1280, 720))
running = True

while running:
    events = pygame.event.get()
    for event in events:
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            print("key pressed")
            print(f"    type(event.key): {type(event.key)}")
            print(f"    event.key: {event.key}")
            print(f"    type(event.mod): {type(event.mod)}")
            print(f"    event.mod: {event.mod}")

pygame.quit()
```

Output from pressing "q"

```
key pressed
  type(event.key): <class 'int'>
  event.key: 113
  type(event.mod): <class 'int'>
  event.mod: 0
```

- How do we know 113 is "q"?
- Pygame has defined constants:

<https://www.pygame.org/docs/ref/key.html#key-constants-label>



Handling KEYDOWN Events

```
import pygame

pygame.init()
screen = pygame.display.set_mode((1280, 720))
running = True

while running:
    events = pygame.event.get()
    for event in events:
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_ESCAPE, pygame.K_q):
                running = False

pygame.quit()
```

- Event loop
- If this is a key down event
 - Check `key` property of event
 - Quit when "q" key pressed
 - Quit when ESCAPE key pressed
 - Combine cases

Time to Clean Up

- Clean up code as you go
- Move event handling to a function
- Main loop looks cleaner now
- Use class so no global variables

```
import pygame

class Game:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((1280, 720))
        self.running = True

    def handle_events(self):
        events = pygame.event.get()
        for event in events:
            if event.type == pygame.QUIT:
                return False
            elif event.type == pygame.KEYDOWN:
                if event.key in (pygame.K_ESCAPE, pygame.K_q):
                    return False

        return True

    def run(self):
        while self.running:
            self.running = self.handle_events()

        pygame.quit()

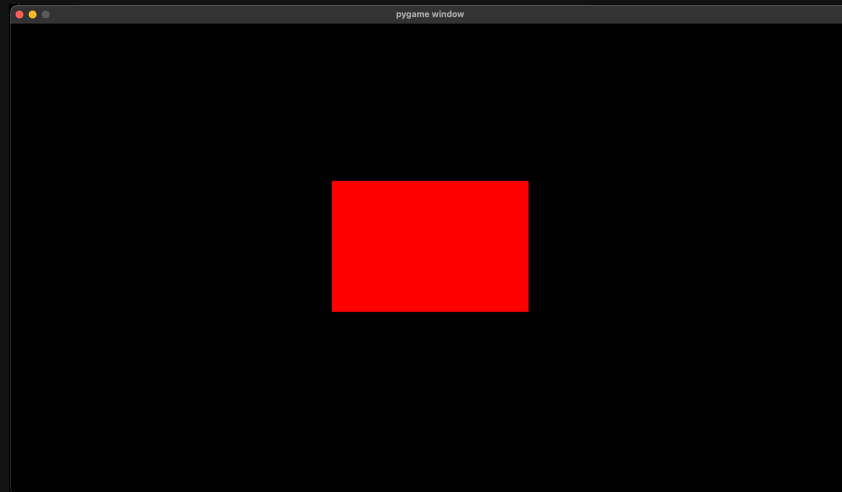
game = Game()
game.run()
```

Summary

- We'll always have an event loop
 - Sometimes simple:
 - Just process events to quit
 - Sometimes complex:
 - Process many keys
 - Process mouse events
 - Process screen resize events
 - Process custom events
- Use classes!
- Next up: drawing

Drawing

- Goal: Draw a red rectangle
- What we need:
 - `pygame.draw.rect(surface, color, rect)`
 - `surface` : what to draw on (e.g., screen)
 - `color` : what color to draw
 - `rect` : rectangle shape and position

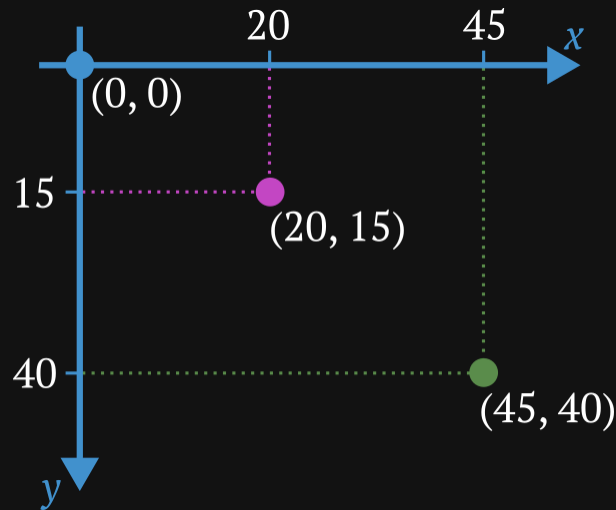


- Let's start with screen coordinates...

Pygame drawing documentation:
<https://www.pygame.org/docs/ref/draw.html>

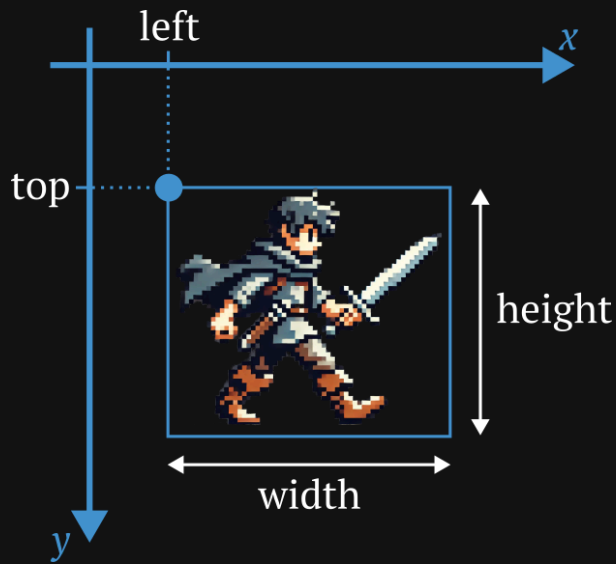
Coordinates

- Need to tell Pygame where to draw things
- Specified by x and y screen coordinates
- Top left of screen is $(0, 0)$
- x axis points to the right
- y axis points down
- Common convention in computer graphics



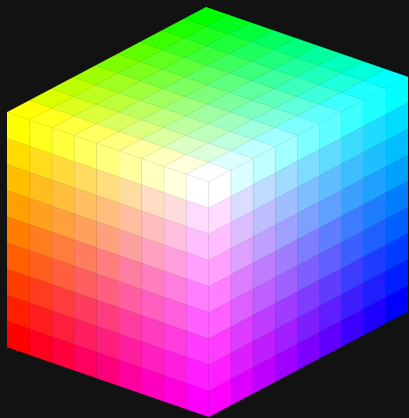
🐍 Rectangles (Rects)

- Fundamental object in Pygame
 - Drawing surface sizes
 - Bounding boxes of sprites
- Usually specified as tuple
 - `(left, top, width, height)`
- Can also be specified as tuple of tuples
 - `((left, top), (width, height))`
 - Convenient when using points `(x, y)`



Colors

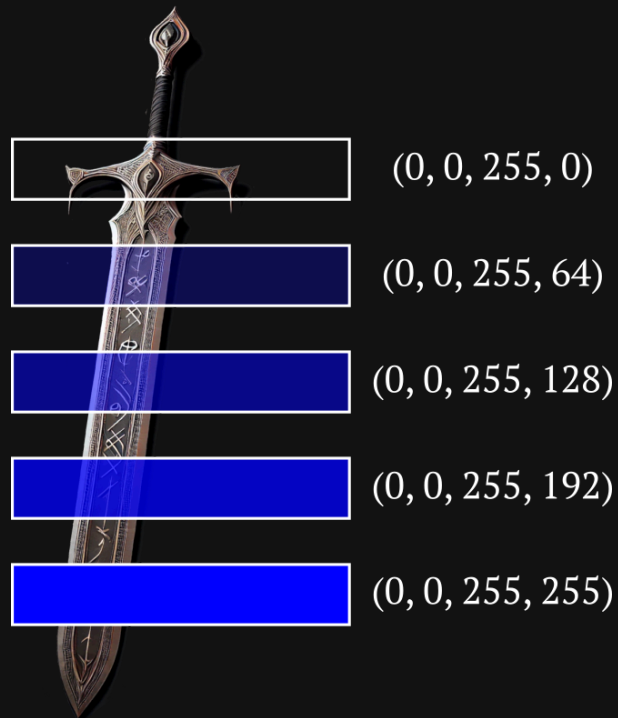
- Color usually specified as RGB tuple
- RGB stands for red, green, and blue
- Each component is between 0-255 (int or float)
- Combine red, green, and blue to get any color



- **red:** (255, 255, 0)
- **green:** (0, 255, 0)
- **blue:** (0, 0, 255)
- **cyan:** (0, 255, 255)
- **magenta:** (255, 0, 255)
- **yellow:** (255, 255, 0)
- **purple:** (126, 30, 156)
- **pink:** (255, 129, 224)
- **light blue:** (149, 208, 252)
- **brown:** (101, 55, 0)
- **grey:** (146, 149, 145)
- **white:** (255, 255, 255)

🐍 Color Transparency

- Color tuple can contain fourth value
- Alpha value (aka, transparency): 0-255
 - 0: completely transparent
 - 255: completely opaque
- Notice colors in illustration
 - All same shade of blue: (0, 0, 255)
 - Different transparencies



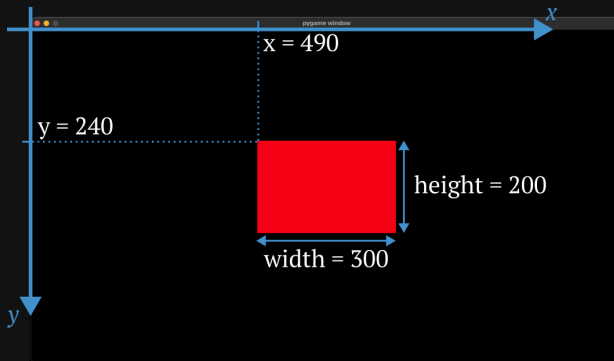


Drawing a Rectangle

```
class Game:
    # ...

    def draw(self):
        self.screen.fill((0, 0, 0))
        pygame.draw.rect(self.screen, (255, 0, 0), (490, 240, 300, 200))
        pygame.display.flip()

    def run(self):
        while running:
            running = handle_events()
            self.draw()
```



- Draw a rectangle
 - Drawing surface: `screen`
 - Color: `(255, 0, 0)`
 - Rect: `(490, 240, 300, 200)`
- What is `flip` ?

🐍 Double Buffering

- Ever make a flip-book?
- Rendering computer graphics is similar
- We draw to one *buffer*
- While showing the other
- Then we *flip* them
- In Pygame, we do it with
`pygame.display.flip()`
- Remember to erase previous drawing



```
def draw(self):  
    self.screen.fill((0, 0, 0)) # erase screen  
    pygame.draw.rect(self.screen, (255, 0, 0), (490, 240, 300, 200))  
    pygame.display.flip()
```

🐍 Moving the Rectangle

```
def __init__(self):
    # ...
    self.x = 490
    self.dx = 1

def draw(self):
    self.screen.fill((0, 0, 0)) # erase screen
    pygame.draw.rect(self.screen, (255, 0, 0), (self.x, 240, 300, 200))
    pygame.display.flip()

def update(self):
    self.x += self.dx
    if self.x >= 980:
        self.dx = -1
    elif self.x <= 0:
        self.dx = 1

def run(self):
    while self.running:
        self.running = self.handle_events()
        self.update()
        self.draw()

pygame.quit()
```



- `update` method
 - updates game state
- `draw` method
 - draws game state to screen
- Making these separate
 - Keeps code cleaner
 - Makes code easier to understand
 - "Separation of concerns"



Smoother motion

```
def __init__(self):  
    # ...  
    self.clock = pygame.time.Clock()  
    self.dt = 0 # elapsed time between frames, in seconds  
  
def run(self):  
    self.dt = self.clock.tick(30) * 0.001 # get elapsed time in seconds
```

- Use Pygame clock
 - Set target frame rate
 - Measure time between frames
 - Gives consistent performance across different devices



2-Dimensional Lists

- Also called "list of lists"
- Let's start by looking at 1-dimensional lists

```
list_0 = ['blue', 'red', 'yellow']
```

```
print(list_0[0]) # blue  
print(list_0[1]) # red  
print(list_0[2]) # yellow
```

- We can imagine the list as a drawer with slots

```
list_0 = 

| 0      | 1     | 2        |
|--------|-------|----------|
| 'blue' | 'red' | 'yellow' |


```




2-Dimensional Lists, continued

```
# three lists
```

```
list_0 = ['blue', 'red', 'yellow']
```

```
list_1 = ['orange', 'green', 'cyan']
```

```
list_2 = ['magenta', 'pink', 'black']
```

	0	1	2
list_0 =	'blue'	'red'	'yellow'

	0	1	2
list_1 =	'orange'	'green'	'cyan'

	0	1	2
list_2 =	'magenta'	'pink'	'black'

```
print(list_0[2]) # yellow
print(list_1[1]) # green
print(list_2[0]) # magenta
```

```
# one 2-dimensional list
```

```
list = [
    ['blue', 'red', 'yellow'],
    ['orange', 'green', 'cyan'],
    ['magenta', 'pink', 'black'],
]
```

0	<table><thead><tr><th>0</th><th>1</th><th>2</th></tr></thead><tbody><tr><td>'blue'</td><td>'red'</td><td>'yellow'</td></tr></tbody></table>	0	1	2	'blue'	'red'	'yellow'
0	1	2					
'blue'	'red'	'yellow'					
1	<table><thead><tr><th>0</th><th>1</th><th>2</th></tr></thead><tbody><tr><td>'orange'</td><td>'green'</td><td>'cyan'</td></tr></tbody></table>	0	1	2	'orange'	'green'	'cyan'
0	1	2					
'orange'	'green'	'cyan'					
2	<table><thead><tr><th>0</th><th>1</th><th>2</th></tr></thead><tbody><tr><td>'magenta'</td><td>'pink'</td><td>'black'</td></tr></tbody></table>	0	1	2	'magenta'	'pink'	'black'
0	1	2					
'magenta'	'pink'	'black'					

```
print(list[2][1]) # pink
print(list[0][2]) # yellow
print(list[1][0]) # orange
```

2-Dimensional Lists, continued

- Indices often referred to as row and column
- Sometimes referred to as y, x or x, y
- Common mistake to mix up your row and column
- (or x and y)

```
# one 2-dimensional list
list = [
    ['blue', 'red', 'yellow'],
    ['orange', 'green', 'cyan'],
    ['magenta', 'pink', 'black'],
]
```

	0	1	2
0	'blue'	'red'	'yellow'
1	'orange'	'green'	'cyan'
2	'magenta'	'pink'	'black'

```
print(list[2][1]) # pink
print(list[0][2]) # yellow
print(list[1][0]) # orange
```



Use Cases for 2-Dimensional Lists

- Game boards
- Pixels in an image
- Grids of buttons (or anything)
- Can treat as list of lists when convenient
 - Use list methods, e.g., `append` , `join` , etc.
- Can use row, column or x,y coordinates when convenient

Homework

- Screen size: 1280 px x 720 px
- Create 2-D list of integers
 - 128 rows x 72 columns, initialize to all zeros
- In update function,
 - For each cell:
 - Set to random value between 0 and 3
- In draw function
 - For each cell:
 - If cell value is 0:
 - Don't do anything
 - Else:
 - Figure out rect coordinates for this cell
 - Draw rect
 - Colors: 1 = red, 2 = blue, 3 = green

